

MODUL PRAKTIKUM

ALJABAR MATRIKS

(PENGENALAN PYTHON I)

Materi:

- 1. Variabel, Tipe Data dan Sistem Bilangan*
- 2. Kontrol Seleksi (If-Else)*

Daftar Isi

Bab 1 Variabel, Tipe Data, dan Sistem Bilangan

- 1.1 Statement dan Expression
- 1.2 Variabel, Objek, dan Tipe Data
- 1.3 Sistem Bilangan dan Operator
- 1.4 Tugas Praktikum 1

Bab 2 Kontrol Seleksi (If-Else)

- 2.1 Boolean Expression
- 2.2 Pernyataan if
- 2.3 Pernyataan if-else
- 2.4 Pernyataan if-elif-else
- 2.5 Pernyataan Bersarang (Nested) dan Pengulangan while
- 2.6 Tugas Praktikum 2

BAB 1. Variabel, Tipe Data, dan Sistem Bilangan

Tujuan Praktikum

- Mahasiswa mampu membedakan statement dan expression dalam Python.
- Mahasiswa mampu membuat, menetapkan ulang, dan menampilkan nilai variabel dengan berbagai tipe data.
- Mahasiswa memahami aturan penamaan variabel yang valid di Python.
- Mahasiswa mampu menggunakan operator aritmatika serta fungsi-fungsi pada modul math, cmath, dan random.

1.1 Statement dan Expression

Statement merupakan perintah atau pernyataan dalam Python. Saat statement dijalankan, tidak ada output yang ditampilkan atau dianggap tidak mengembalikan nilai apa pun.

Kode 1.1

```
my_int = 5
print(my_int)
```

Expression merupakan kombinasi nilai dan operasi yang menciptakan nilai baru, disebut return value, yaitu nilai yang dikembalikan oleh operasi tersebut.

Kode 1.2

```
print(my_int + 5)
```

Kode 1.3

```
x = 1
print(x + 5)

y = x + 5
print(y)
```

1.2 Variabel, Objek, dan Tipe Data

Variable adalah nama yang dibuat untuk mewakili “sesuatu” dalam program, dapat berupa bilangan, kumpulan data, file, string, sekuen, array, dan sebagainya. Variabel pada Python ditulis dengan menambahkan tanda sama dengan (=) setelah nama variabel, diikuti nilai yang diberikan.

Kode 1.4

```
import math
x = 5
```

```
y = math.pi + 5
z = "Hello World"
print(x)
print(y)
print(z)
print(A)    # <- error: variabel A belum didefinisikan
```

Perhatikan

Baris terakhir kode 1.4 akan menghasilkan NameError karena variabel A belum pernah didefinisikan sebelumnya. Ini adalah kesalahan umum yang perlu diwaspadai.

Jika sebuah variabel yang sudah ada diberi nilai baru, maka nilai baru tersebut akan menimpa (overwrite) nilai lama.

Kode 1.5

```
n = 4
print(n)

n = "empat"
print(n)
```

Python juga memungkinkan penulisan beberapa variabel sekaligus dalam satu baris (multiple assignment):

Kode 1.6

```
a, b = 1, 2
print(a)
print(b)
print(a + b)
```

Variabel juga dapat diperoleh dari input pengguna melalui fungsi `input()`, yang secara default selalu mengembalikan nilai bertipe string.

Kode 1.7

```
print('Enter your name:')
x = input()
print("Hello," + x)
```

Kode 1.8

```
x = input('Enter your name:')
print("Hello," + x)
```

Kode 1.9 — mengonversi input menjadi integer

```
y = int(input('How old are you?'))
print(y)
print(type(y))
```

Studi Kasus: Menghitung Body Mass Index (BMI)

Body Mass Index (BMI) adalah angka yang dihitung dari berat dan tinggi badan seseorang, digunakan sebagai indikator kegemukan tubuh. Rumus BMI adalah berat (kg) dibagi kuadrat tinggi (meter):

Kode 1.10 — Program BMI

```
import math

print("Masukkan berat : (kg)")
berat = float(input())
print("Masukkan Tinggi : (m)")
tinggi = float(input())
Bmi = berat / tinggi**2

print(Bmi)
```

Aturan Penulisan Nama Variabel

Terdapat beberapa aturan wajib dalam menamai variabel di Python:

- Tidak boleh diawali dengan angka. Contoh benar: data1, _angka. Contoh salah: 1data, 9nilai.
- Hanya boleh mengandung huruf, angka, dan underscore (_). Contoh benar: data_siswa, nilai_ujian1. Contoh salah: data-siswa, nilai@ujian.
- Tidak boleh menggunakan kata kunci (keyword) Python, seperti if, while, for, True, False, dll.
- Bersifat case-sensitive — huruf besar dan huruf kecil dianggap berbeda (nama ≠ Nama).

Kode 1.11 — contoh nama variabel tidak valid

```
rich&bill = 'sasa'
print(rich&bill)
# Program di atas akan ERROR karena '&' bukan karakter yang diperbolehkan pada
nama variabel
```

Setiap objek pada Python memiliki atribut, salah satunya adalah tipe data (dapat dicek dengan fungsi `type()`) dan identitas objek di memori (dapat dicek dengan fungsi `id()`).

Kode 1.12

```
a_int = False
print(type(a_int))
print(id(a_int))
```

Kode 1.13

```
b_float = 2.5
print(type(b_float))
print(id(b_float))
```

1.3 Sistem Bilangan dan Operator

Python mendukung operator aritmatika berikut untuk bilangan integer dan float:

- + (penjumlahan)
- - (pengurangan)
- * (perkalian)
- / (pembagian)
- ** (eksponen / pemangkatan)
- % (modulus / sisa bagi)
- // (floor division / pembagian bulat ke bawah)

Kode 1.14 — Operasi Aritmatika

```
# OPERASI ARITMATIKA
a = 9
b = 2

hasil = a + b
print(a, "+", b, "=", hasil)

hasil = a - b
print(a, "-", b, "=", hasil)

hasil = a * b
print(a, "*", b, "=", hasil)

hasil = a / b
print(a, "/", b, "=", hasil)

hasil = a ** b
print(a, "**", b, "=", hasil)

hasil = a % b
print(a, "%", b, "=", hasil)

hasil = a // b
print(a, "//", b, "=", hasil)
```

Prioritas Operasi (Operator Precedence)

1. Tanda kurung ()
2. Perkalian, pembagian, modulus, floor division: * / % //
3. Penjumlahan dan pengurangan: + -

Beberapa fungsi matematika berguna tersedia pada modul math (perlu import math terlebih dahulu):

Kode 1.15 — Fungsi modul math

```
import math
print(math.floor(-3.4))
print(math.ceil(-3.4))
print(math.pow(9, 0.5))
print(math.sqrt(9))
print(math.exp(2))
print(math.log(math.e))
print(math.log(101, 10))
print(math.sin(math.pi * 1.3))
print(math.atan2(3, 4))
```

Untuk operasi bilangan kompleks, gunakan modul `cmath`. Untuk menghasilkan bilangan acak (random), gunakan modul `random`.

Kode 1.16 — modul cmath

```
import cmath
print(cmath.sqrt(-9))
print(cmath.exp(math.pi * 1j) + 1)
```

Kode 1.17 — modul random

```
import random
print(random.random())
print(random.random())
```

Latihan pemahaman: hitung secara manual hasil dari ketiga ekspresi berikut sebelum menjalankan kodenya, lalu perhatikan bagaimana operator `**` dievaluasi (dari kanan ke kiri / right-associative).

Kode 1.18

```
A = 2**2**3
B = 2**(2**3)
C = (2**2)**3

print(A, B, C)
```

1.4 Tugas Praktikum 1

Soal Latihan 1

Manakah dari nama-nama berikut yang merupakan nama variabel yang dapat diterima (valid) pada Python? Jelaskan alasan untuk setiap nama yang tidak valid.

```
xy_zzy = 5
2ndVar = 5
rich&bill = 5
long name = 5
long-name = 5
good2go = 5
```

```
_age = 5  
var = 5  
age_ = 5  
box1 = 5
```

Soal Latihan 2

1. Diberikan `weight = 65` dan `height = 177`, mengapa `print(Weight)` akan menghasilkan error? (Perhatikan sifat case-sensitive Python.)
2. Diberikan `x, y = 2, 6` lalu `x, y = y, x + 2`. Berapa nilai akhir `x` dan `y`? Ingat bahwa Python selalu mengevaluasi seluruh sisi kanan terlebih dahulu sebelum melakukan assignment ke sisi kiri.
3. Tuliskan tipe data dan hasil `id()` dari variabel `A, B, C, D, E, F, G, H, I, J` berikut, lalu jelaskan mengapa `E` dan `A` memiliki id yang sama.

```
A = 5  
B = 50.0  
C = 50  
D = '23yearsold'  
E = 5  
F = D  
G = 567_100  
H = 1e14  
I = True  
J = 1000-25
```

Soal Latihan 3

Buatlah program yang meminta input berat badan (kg) dan tinggi badan (m) dari pengguna, lalu menghitung dan menampilkan nilai BMI. Modifikasi Kode 1.10 agar hasil BMI ditampilkan dengan 2 angka di belakang koma.

BAB 2. Kontrol Seleksi (If-Else)

Tujuan Praktikum

- Mahasiswa memahami konsep kontrol seleksi (control flow) dan ekspresi Boolean dalam Python.
- Mahasiswa mampu menerapkan struktur if, if-else, dan if-elif-else untuk pengambilan keputusan.
- Mahasiswa mampu menyusun struktur kontrol seleksi bersarang (nested) untuk kasus dengan banyak kondisi.
- Mahasiswa mengenal penggunaan pengulangan while sebagai pelengkap kontrol alur program.

Dalam pemrograman, kontrol seleksi adalah proses penerapan keputusan untuk mengontrol pilihan: melewati beberapa pernyataan, mengeksekusi serangkaian pernyataan secara berulang, atau memilih di antara kumpulan pernyataan alternatif untuk dieksekusi. Struktur kontrol seleksi pada Python direpresentasikan dengan pernyataan if-elif-else.

Kontrol seleksi berkaitan erat dengan ekspresi Boolean bernilai True atau False. Jika pernyataan bernilai True, blok kode akan dieksekusi; jika False, blok tidak dieksekusi (atau dialihkan ke blok alternatif). Setelah seleksi selesai, program dilanjutkan mengeksekusi pernyataan berikutnya secara teratur.

2.1 Boolean Expression

Python menyediakan operator pembandingan (kondisional) yang hasilnya selalu berupa nilai boolean, True atau False:

- `==` (sama dengan)
- `!=` (tidak sama dengan)
- `<` (kurang dari)
- `>` (lebih besar dari)
- `<=` (kurang dari atau sama dengan)
- `>=` (lebih besar dari atau sama dengan)

Kode 2.1 — operator logika and / or / not

```
print(2 < 3 and 3 > 1)
print(2 < 3 or 3 > 1)
print(2 < 3 or not 3 > 1)
print(2 < 3 and not 3 > 1)
```

Kode 2.2

```
x = 4
y = 5
```

```
p = x < y or x < z    # <- error: variabel z belum didefinisikan
print(p)
```

Perhatikan

Kode 2.2 akan menghasilkan `NameError` karena variabel `z` tidak pernah didefinisikan sebelumnya. Python tetap mengevaluasi seluruh ekspresi meskipun bagian pertama (`x < y`) sudah bernilai `True`, kecuali jika `short-circuit evaluation` membuatnya tidak perlu dievaluasi — namun di sini keduanya dihubungkan dengan `or` dan `x < y` sudah `True`, sehingga sebenarnya `x < z` tidak akan dievaluasi. Cobalah jalankan sendiri untuk memverifikasi perilaku ini.

Kode 2.3 — hati-hati menimpa nama bawaan Python

```
True, False = False, True    # menimpa nilai bawaan True dan False!
print(True, False)
print(2 < 3)
```

2.2 Pernyataan if

Pernyataan `if` adalah cara melakukan pengambilan keputusan yang memungkinkan eksekusi kondisional dari suatu pernyataan berdasarkan nilai ekspresi. Struktur `if` paling sederhana:

Kode 2.4

```
nilai = 59
if nilai >= 60 :
    print("Selamat kamu lulus Pemograman Dasar")
print('asoooy')
```

Perhatikan bahwa `print('asoooy')` tetap dijalankan meskipun kondisi `if` bernilai `False`, karena baris tersebut berada di luar blok `if` (tidak memiliki indentasi ekstra).

Kode 2.5 — menggunakan f-string

```
nilai = 85
Mk_Lulus = 0
if nilai >= 60:
    Mk_Lulus += 1
    print(f"Nilai Pemrogrman Dasar Kamu {nilai}")
    print("Selamat kamu lulus Pemograman Dasar")
print("Tetap Semangat")
```

Pernyataan `if` digunakan untuk mengeksekusi sepotong kode hanya ketika ekspresi boolean bernilai benar (`True`):

Kode 2.6

```
x = 42
if x % 2 == 0:
    print('even')
```

2.3 Pernyataan if-else

Terkadang, saat mengevaluasi suatu kondisi, kita perlu mengambil satu pilihan jika kondisi benar dan menyediakan pilihan alternatif jika kondisi salah. Inilah fungsi klausa if-else.

Kode 2.7

```
nilai = 50
Mk_Lulus = 0
Mk_Belum_lulus = 0
if nilai >= 60 :
    Mk_Lulus += 1
    print("Nilai Pemograman Dasar Kamu %d" %(nilai))
    print("Selamat kamu lulus Pemograman Dasar")
    print("Jumlah MK LULUS = %d" %(Mk_Lulus))
else:
    Mk_Belum_lulus += 1
    print("Nilai Pemograman Dasar Kamu %d" %(nilai))
    print("Kamu Belum lulus Pemograman Dasar")
    print("Jumlah MK MENGULANG = %d" %(Mk_Belum_lulus))
    print("Silakan mengulang Pemograman Dasar di Semester Depan")
    print("SEMANGAT !!")
```

2.4 Pernyataan if-elif-else

Jika if dan if-else hanya mengevaluasi satu ekspresi boolean, klausa elif (kependekan dari “else if”) memungkinkan pengecekan beberapa kondisi sekaligus secara berurutan. Struktur ini cocok digunakan ketika keputusan diambil dari beberapa kemungkinan kondisi.

Kode 2.8 — dua blok if terpisah (bukan elif), keduanya dievaluasi independen

```
a = 2
if a < 10 :
    print('a')
if a < 5 :
    print('b')
```

Kode 2.9 — konversi nilai angka menjadi grade huruf

```
nilai = 79
Mk_Lulus = 0
Mk_Belum_lulus = 0
if nilai >= 80 :
    Mk_Lulus += 1
    print("Nilai Pemograman Dasar Kamu %d (A)" %(nilai))
    print("Selamat kamu lulus Pemograman Dasar")
elif nilai >= 70 :
    Mk_Lulus += 1
    print("Nilai Pemograman Dasar Kamu %d (B)" %(nilai))
    print("Selamat kamu lulus Pemograman Dasar")
elif nilai >= 60 :
    Mk_Lulus += 1
```

```

    print("Nilai Pemograman Dasar Kamu %d (C)" %(nilai))
    print("Selamat kamu lulus Pemograman Dasar")
else:
    Mk_Belum_lulus += 1
    print("Kamu belum lulus, semangat mengulang!")

```

Kode 2.10

```

x = 42
if x < 10:
    print('one digit number')
elif x < 100:
    print('two digit number')
else:
    print('big number')

```

2.5 Pernyataan Bersarang (Nested) dan Pengulangan while

Struktur if-elif-else dapat disarangkan (nested), yaitu diletakkan di dalam blok if/else lainnya, untuk menangani kombinasi lebih dari satu kondisi sekaligus.

Kode 2.11 — nested if: rekomendasi menu

```

suka_pedas = input("Suka pedas (Y/T)? ")
tanggal_tua = input("Tanggal tua (Y/T)? ")
if suka_pedas == "Y":
    if tanggal_tua == "Y":
        print("Rekomendasi menu: Nasi sambal")
    else:
        print("Rekomendasi menu: Nasi rica-rica iga sapi")
else:
    if tanggal_tua == "Y":
        print("Rekomendasi menu: Nasi kecap")
    else:
        print("Rekomendasi menu: Nasi ayam kecap")

```

Kode 2.12 — nested if dengan beberapa kondisi bertingkat

```

var = float(input("var = "))
if var % 2 :
    if var **3 != 27:                # (bilangan ganjil selain 3)
        var = var + 4
    else:                            # (3)
        var /= 1.5
else :
    if var <= 10:                    # (bilangan genap <= 10)
        var *= 2
    else :                           # (bilangan genap > 10)
        var -= 2
print(var)

```

Selain kontrol seleksi, Python juga menyediakan pengulangan while, yang akan terus mengeksekusi blok kode selama kondisinya bernilai True. Contoh berikut memaksa pengguna memasukkan angka positif sebelum program dapat melanjutkan:

Kode 2.13 — validasi input dengan while

```
x = int(input("Masukan angka: \n"))
while x < 0:
    print("Masukan angka positif")
    x = int(input("Masukan angka: \n"))
print(f"{x}")
```

2.6 Tugas Praktikum 2

4. Apa yang terjadi ketika kode berikut dijalankan? Apakah akan memberikan kesalahan (error)? Jelaskan alasannya.

```
x = 9
if x == 2:
    print(x)
else:
    print(y)    # <- variabel y belum pernah didefinisikan
```

5. Apa yang terjadi ketika kode berikut dijalankan? Apakah akan memberikan kesalahan?

```
x = 2
if x == 2:
    print(x)
else:
    x += 2
# Setelah blok if-else selesai, apakah ada print() yang menampilkan nilai x?
```

6. Tulis program yang meminta input sebuah angka dan memeriksa apakah bilangan tersebut positif, negatif, atau nol. Cetak 'Positif' jika bilangan lebih besar dari 0, 'Negatif' jika kurang dari 0, dan 'Nol' jika sama dengan 0.
7. Modifikasi program pada Kode 2.13 (validasi angka positif dengan while) agar setelah mendapatkan angka positif, program juga mencetak nilai absolut dari angka tersebut menggunakan fungsi abs().
8. Buatlah program konversi nilai ujian (0–100) menjadi grade huruf A, B, C, D, atau E menggunakan struktur if-elif-else, dengan ketentuan: A (≥ 85), B (70–84), C (60–69), D (50–59), E (< 50).